
REPUBLIC OF
GAMERS



Introduction

Requirements:
Unity IDE version: 2018.4.19f1 or above.
Android Min API Level: 19 (Android 4.4)

ROG Phone SDK provides game developers easy ways to unleash the full capabilities of the ROG Phone. We provide 5 key functions in our ROG Phone SDK. It allows developers to communicate with ROG TwinView Dock, ROG Kunai Gamepad and Aura lighting on the ROG Phone. Also, ROG Phone SDK allows developers to dynamically adjust the refresh rate and performance of ROG Phone to achieve the best control for game development. In the following sections, we'll introduce each function one by one, followed by an installation guide and how to use them.

ROG TwinView Dock

Requirements: ROG II TwinView Dock or above, OpenGL ES

The whole-new ROG TwinView Dock provides a genuine smooth gaming experience on both phone display and extended display. This feature provides dual display support for games to show more information. For example, you can play Shadowgun Legends with controls at the lower screen, and gameplay view at the upper screen.



Figure 1. Playing Shadowgun Legends with ROG Phone II and ROG II TwinView Dock

ROG Kunai Gamepad

Requirements: ROG II Kunai Gamepad or above

ROG Kunai Gamepad provides console-style physical controls on mobile gaming, including the games you are working on.
You can configure to map ROG Kunai Gamepad controls to the events in games as you want through ROG Phone SDK. That is, your game will be natively supported by ROG Kunai Gamepad without any extra configuration by users.



Figure 2. Playing Shadowgun Legends on ROG Phone II with ROG Kunai Gamepad

Refresh rate control

Requirements: ROG Phone II or above

ROG Phone takes gaming visuals to the next level with the world's highest display quality, which makes mobile gaming an absolute dream. Combined with the world-learning response time and touch sampling rate, the ROG Phone easily delivers one of the smoothest gaming experiences you'll have on an Android smartphone. For developers, they can dynamically control the refresh rate of the display on ROG Phones after ROG Phone II through ROG Phone SDK, to fulfill their needs in games.

Aura lighting and ROG Vision

Requirements: ROG Phone II or above

Aura lighting is developed to perfectly complement your gaming rig for a true expression of individuality. Aura lighting provides a nearly endless spectrum of colors, patterns, and even the ability to link lighting to music, in-game action, or system status (e.g. CPU/GPU temperatures). Aura lighting can be presented as ROG logo light or RGB light on the back of ROG Phone. Developers can configure Aura lighting on ROG Phones after ROG Phone II to have different lighting themes (e.g. static, breathing, strobing and color cycling), lighting speed (slow, medium, high), and lighting colors.

In addition, since ROG Phone 5, ROG Vision is introduced as a part of visual elements shown on ROG Phone. ROG Vision is a stylish panel on the back of ROG Phone, to play animated images to correspond to gaming events or system behaviors. That is, ROG Vision provides a higher dimension of visual presentation, and developers can use this panel to express richer messages in games.

Performance boost

Requirements: ROG Phone II or above

Developers can boost system performance for a defined length of time, to handle computing extensive tasks, such as massive model rendering or other tasks to consume computing resources temporarily. Developers can reference performance metrics, such as FPS and CPU status to decide when to boost phone performance when these performance metrics drop, to reach a smooth experience in game.

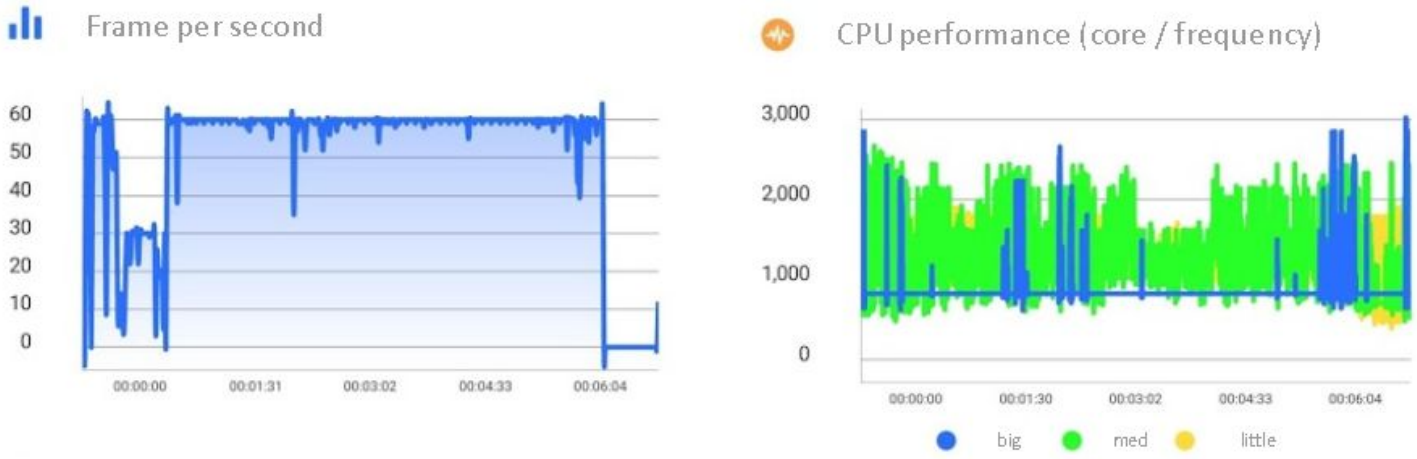


Figure 3. Example of game playing performance by frame per second and CPU status

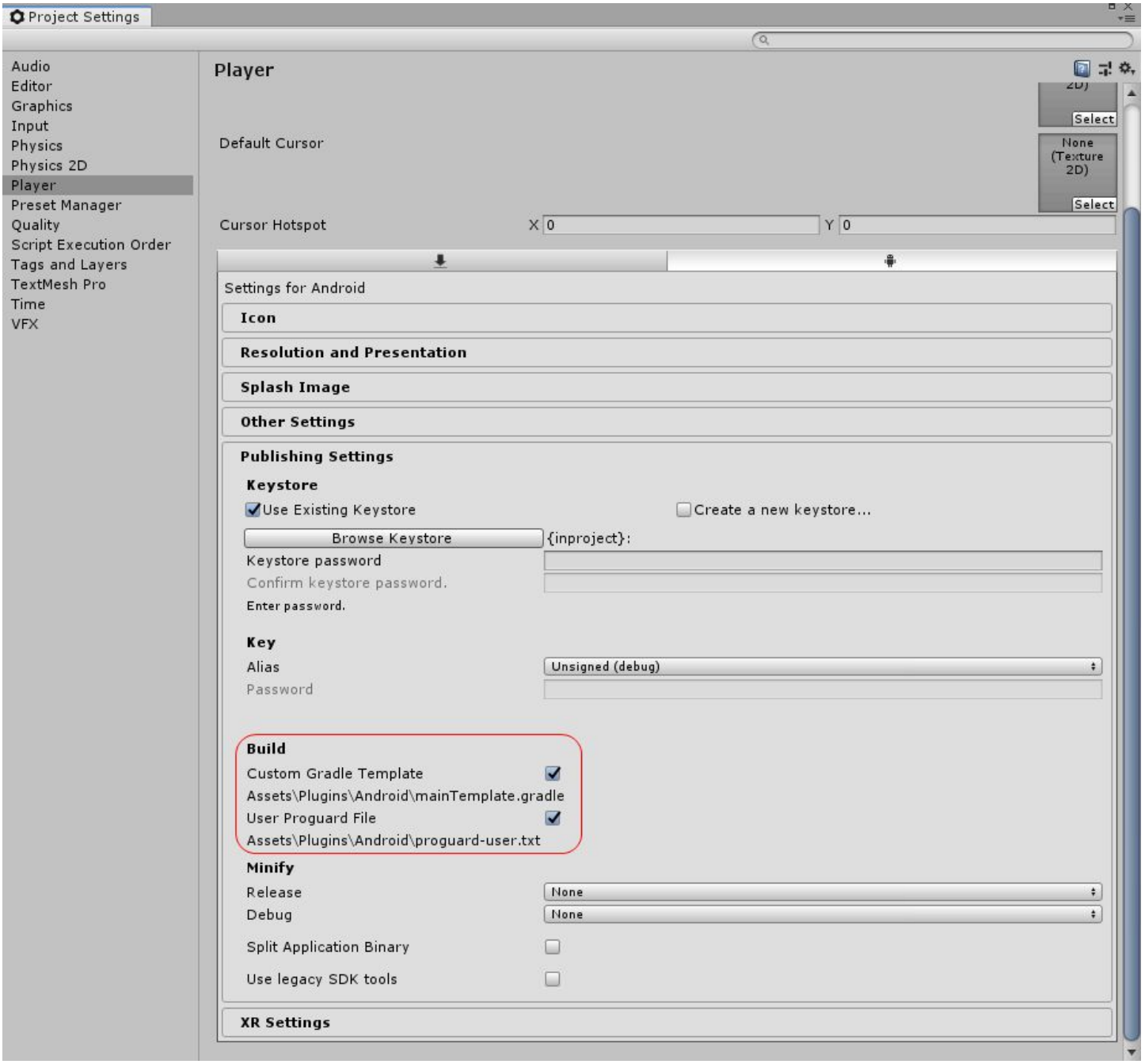
Installation

Installing from Unity Asset Store

Go to “**Unity’s main menu > Window > Asset Store**”, and search for “**ROG Phone SDK**.” Download and Import. When the window “Import Unity package” shows up, select all the content and click Import.

In order to make sure all the SDK dependencies are resolved, please follow steps below (Gradle Integration):

1. Go to “**Build Settings > Player Settings -> Publishing Settings**” and check “**Custom Gradle Template**.”



2. Find the generated template gradle file named “**mainTemplate.gradle**” in “**Assets > SDKs > Android**”

3. Edit the gradle file and add the following lines:

```
dependencies {
        implementation fileTree(dir: 'libs', include: ['*.jar'])
        implementation "org.jetbrains.kotlin:kotlin-stdlib-jdk7:1.3.41”
}
```

You can use sample mainTemplate.gradle and AndroidManifest.xml in Unity Package to build a demo app which is also included in our Unity Package.

Proguard rules

If your project uses proguard minification, please add our proguard rules file into your project.

You can achieve this goal via copying our proguard rules file from **Assets/RogSDK/Documentation/proguard-user.txt** to **Assets/Plugin/Android** folder.

Scripting API: TwinViewManager

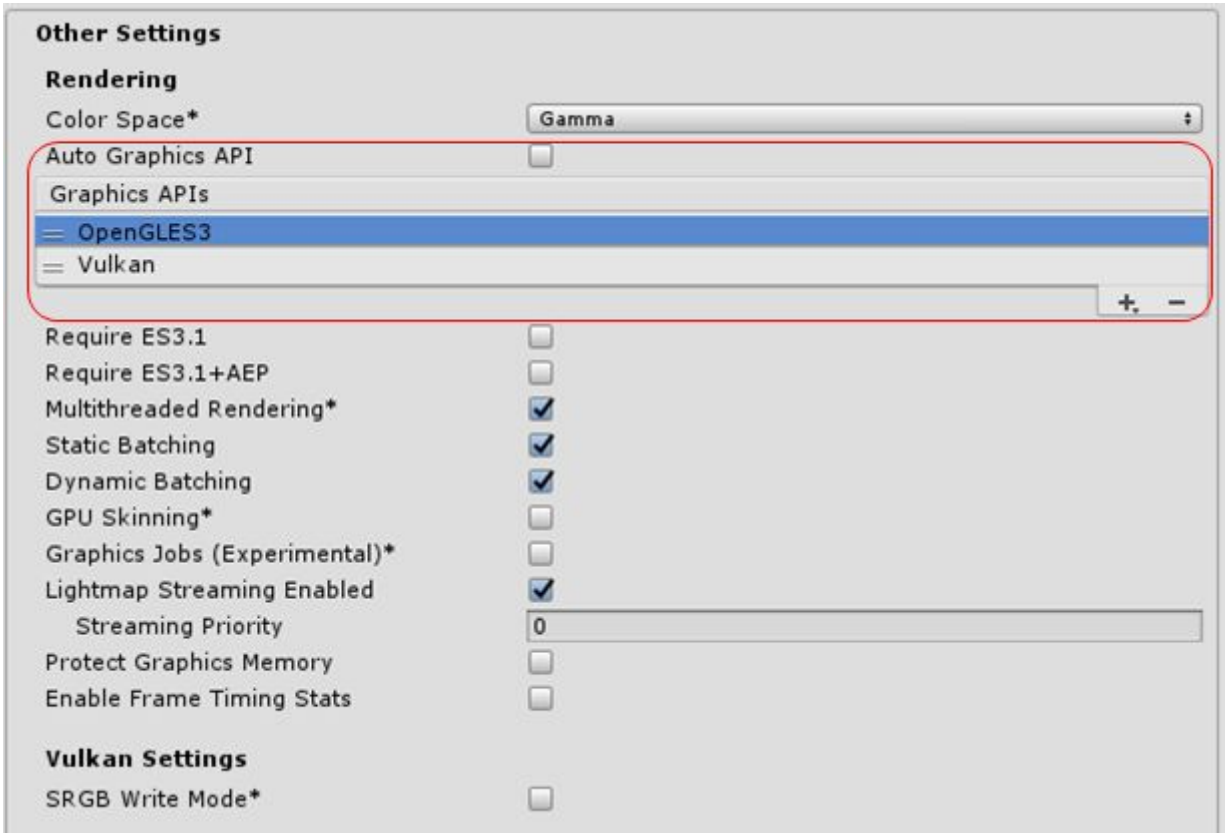
Description

Helper class to show different contents on dual display.

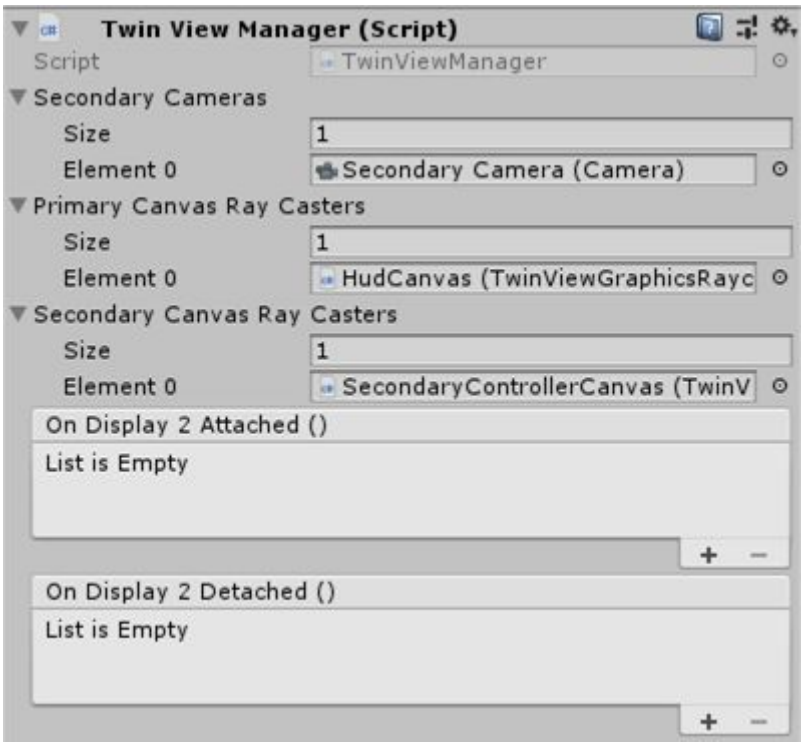
Setup

Based on Unity official announcement, Unity only supports multi-display on Android platform with **OpenGL ES** (Vulkan not). See Unity official document for multi-display on Multidisplay.

You can update your **Graphics Settings** in **Player Settings** and add OpenGLES3/OpenGLES2 into the **Graphics APIs** column. If your application uses Vulkan, you still can add Vulkan into **Graphics APIs**, but make sure the OpenGLES3/OpenGLES2 should list before Vulkan to support our TwinView feature.



To enable TwinView support, please drag the **AsusTwinManager** into your scene. You will see the following in your Inspector shown below, and you should set up the configuration of the secondary display as in the following picture.



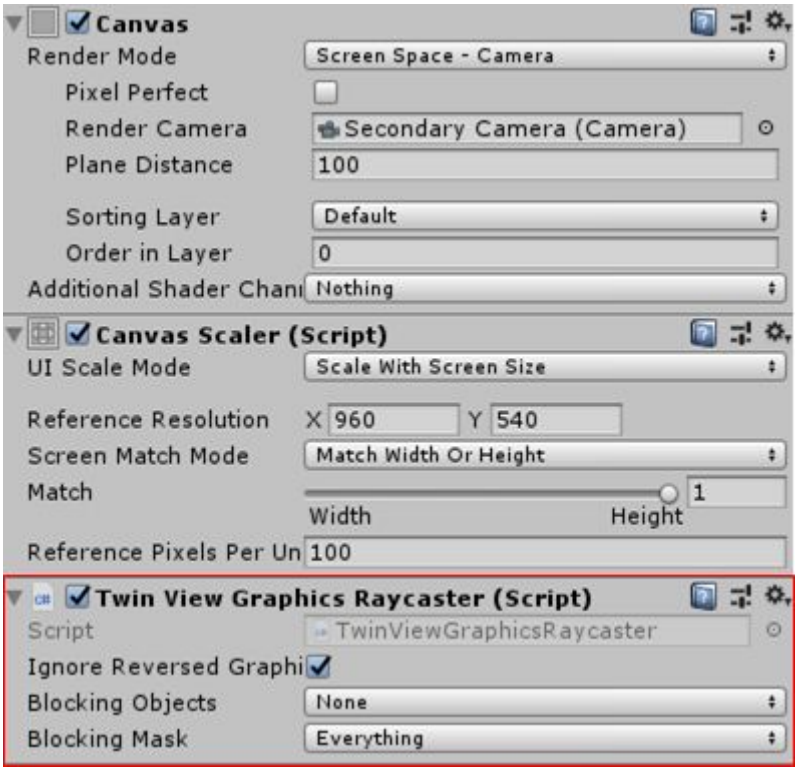
Normally you should have two GameObjects with Canvas components and two GameObjects with Camera components (one Canvas and one Camera for primary display and the other one Canvas and Camera for secondary display) in your scene.

Then attach **TwinViewGraphicsRaycaster.cs** to your GameObject with Canvas component.

Finally please drag the Camera GameObject for secondary display to Secondary Cameras column.

And drag corresponding items to the column of **Primary Canvas Ray Caster** and **Secondary Canvas Ray Caster**.

If you want to get the callback call for the secondary display attached/detached event. Click **+**, and add your GameObject and function that you would like to respond to.



Notice that the name of **AsusTwinViewManger** GameObject should not be changed. You should make sure there is only one GameObject named to **AsusTwinViewManager** in your scene.

Properties

TwinViewGraphicsRaycaster[] **primaryCanvasRayCasters**	List of TwinViewGraphicsRaycaster objects for primary display.
List<Camera> **secondaryCameras**	List of Camera objects for secondary display.
TwinViewGraphicsRaycaster[] **secondaryCanvasRayCasters**	List of TwinViewGraphicsRaycaster objects for secondary display.
UnityEvent **onDisplay2Attached**	Callback event when secondary display attached.
UnityEvent **onDisplay2Detached**	Callback event when secondary display detached.

Public Methods

public void **SwapDisplayContent**()	Switch the content in primary display to secondary display.

Scripting API: Gamepad

Description

Helper class to register ROG Kunai Gamepad key event callback.

Setup

First, please change your app's default Activity to our **GamepadActivity** to support our Gamepad feature.

You can add a new custom AndroidManifest.xml in **Assets/Plugins/Android** to achieve this.

(See Unity official document on UnityPlayerActivity example file with sample manifest)

Your custom AndroidManifest.xml may look like:

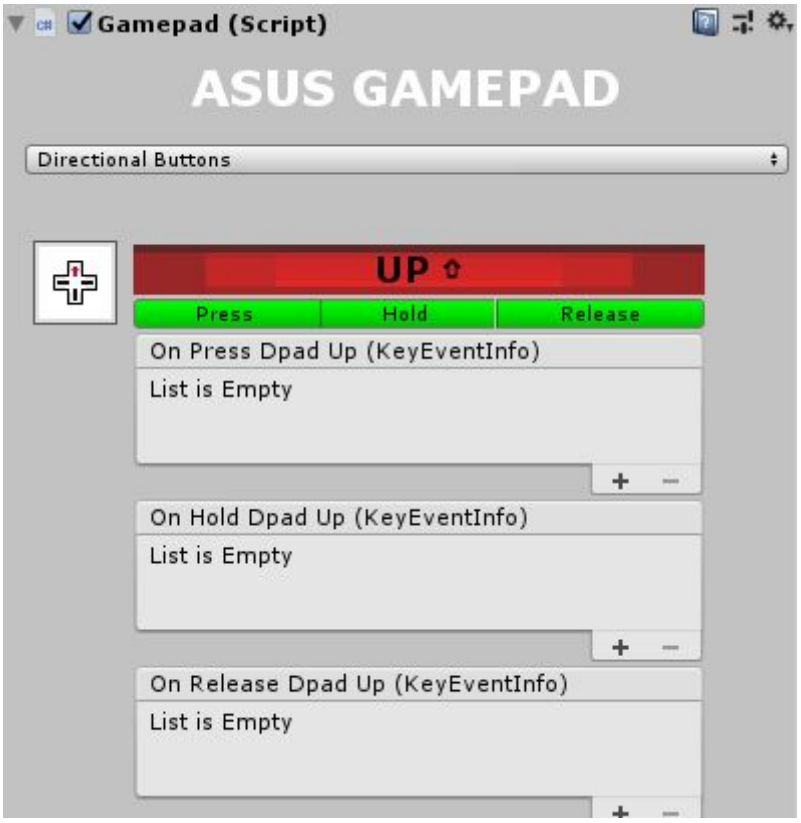
Assets/Plugins/Android/AndroidManifest.xml

```
<manifest ...>
  <application ...>
    <activity android:name="com.asus.rogplugin.gamepad.GamepadActivity">
      <intent-filter>
        <action android:name="android.intent.action.MAIN" />
        <category android:name="android.intent.category.LAUNCHER" />
      </intent-filter>
    </activity>
  </application>
</manifest>
```

To enable Gamepad support, please attach **Gamepad.cs** to your GameObject. You will see the followings in your Inspector shown below, and choose what control type you would like to handle.



For example, if we select **Directional Buttons**, and click "**Press**" for **Up** button, it means we want to handle the press event for directional button-> **UP** (i.e. UP key). Click **+**, and add your GameObject and function that you would like to response press event for the **UP** key.



Properties

ConnectionEvent **onGamepadConnected**	Callback event when a gamepad device is connected.
ConnectionEvent **onGamepadDisconnected**	Callback event when a gamepad device is disconnected.

KeyEvent **onHold*[BUTTON_NAME]***	Callback event when the button *[BUTTON_NAME]* is holding.
KeyEvent **onPress*[BUTTON_NAME]***	Callback event when the button *[BUTTON_NAME]* is pressed.
KeyEvent **onRelease*[BUTTON_NAME]***	Callback event when the previous pressed/holding button *[BUTTON_NAME]* is released.
MotionEvent **onMoveLeftStick**	Callback event for left joystick.
MotionEvent **onMoveRightStick**	Callback event for right joystick.

Public Static Methods

public static string **GetCurrentConnectedDevice**()	Gets all of current connected devices.
public static bool **ShouldEnableGamepadEvent**()	Gets whether you should use the implementation based on Gamepad class or not. When it returns false that means you should use the implementation based on Unity Input System.

Scripting API: ConnectionEvent

Description

Callback event contains **DeviceInfo** when a gamepad device is connected/disconnected.

Scripting API: KeyEvent

Description

Callback event contains **KeyEventInfo** when a button is pressed/holding/released.

Scripting API: MotionEvent

Description

Callback event contains **MotionEventInfo** when the left or right joystick moves.

Scripting API: DeviceInfo

Description

Data class to store the device information.

Properties

int **deviceId**	The input device id.
GamepadEnum **gamepad**	The kind of gamepad for the input device.

Scripting API: EventInfo

Description

Abstract data class to store the event information sent from gamepad.

Properties

int **deviceId**	The input device id that this event came from.
GamepadEnum **gamepad**	The kind of gamepad that this event came from.
InputControlEnum **inputControl**	The kind of control that this event came from.

Enumerations: GamepadEnum

KUNAI_GAMEPAD	Type of ROG Kunai Gamepad.
KUNAI_3_GAMEPAD	Type of ROG Kunai 3 Gamepad.
UNKNOWN	Type of unknown device.

Enumerations: InputControlEnum

BUTTON_EAST	Control representing the **B** button.
BUTTON_NORTH	Control representing the **Y** button.
BUTTON_SOUTH	Control representing the **A** button.
BUTTON_WEST	Control representing the **X** button.
DPAD_DOWN	Control representing the **D-Pad down** key.

DPAD_LEFT	Control representing the **D-Pad left** key.
DPAD_RIGHT	Control representing the **D-Pad right** key.
DPAD_UP	Control representing the **D-Pad up** key.
FUNCTION	Control representing the **Function** button.
LEFT_SHOULDER	Control representing the **L1** button.
LEFT_STICK	Control representing the **left joystick** button.
LEFT_TRIGGER	Control representing the **L2** button.
M1	Control representing the **M1** button.
M2	Control representing the **M2** button.
M3	Control representing the **M3** button.
M4	Control representing the **M4** button.
MODE	Control representing the **Mode** button.
PROFILE	Control representing the **Profile** button.
RIGHT_SHOULDER	Control representing the **R1** button.
RIGHT_STICK	Control representing the right **joystick** button.
RIGHT_TRIGGER	Control representing the **R2** button.
SELECT	Control representing the **Select** button.
START	Control representing the **Start** button.

Enumerations: PhaseEnum

HOLD	Type of holding state.
PRESS	Type of press state.
RELEASE	Type of release state.

Scripting API: KeyEventInfo

Description

Data class which inherits from **EventInfo** to store the key and button event information sent from gamepad.

Properties

PhaseEnum **phaseEnum**	The phase of this key event.

Scripting API: MotionEventInfo

Description

Data class which inherits from **EventInfo** to store the movement event information sent from gamepad.

Properties

Vector2 **axes**	The X and Y axis value of this motion event.

Scripting API: RogKunaiGamepadAndroid

Description

RogKunaiGamepadAndroid is defined as a custom device for Unity Input System. You can use this class to control **ROG Kunai Gamepad** feature buttons when it is connected to an Android device.

See Input System for more details.

Controls

Control	Type	Description
functionButton	ButtonControl	The **Function** button on ROG Kunai Gamepad
m1Button	ButtonControl	The **M1** button on ROG Kunai Gamepad
m2Button	ButtonControl	The **M2** button on ROG Kunai Gamepad
m3Button	ButtonControl	The **M3** button on ROG Kunai Gamepad
m4Button	ButtonControl	The **M4** button on ROG Kunai Gamepad
profileButton	ButtonControl	The **Profile** button on ROG Kunai Gamepad

Scripting API: RogKunai3GamepadAndroid

Description

RogKunai3GamepadAndroid is defined as a custom device for Unity Input System. You can use this class to control **ROG Kunai 3 Gamepad** feature buttons when it is connected to an Android device.

See Input System for more details.

Controls

Control	Type	Description
m1Button	ButtonControl	The M1 button on ROG Kunai 3 Gamepad
m2Button	ButtonControl	The M2 button on ROG Kunai 3 Gamepad
modeButton	ButtonControl	The Mode button on ROG Kunai 3 Gamepad

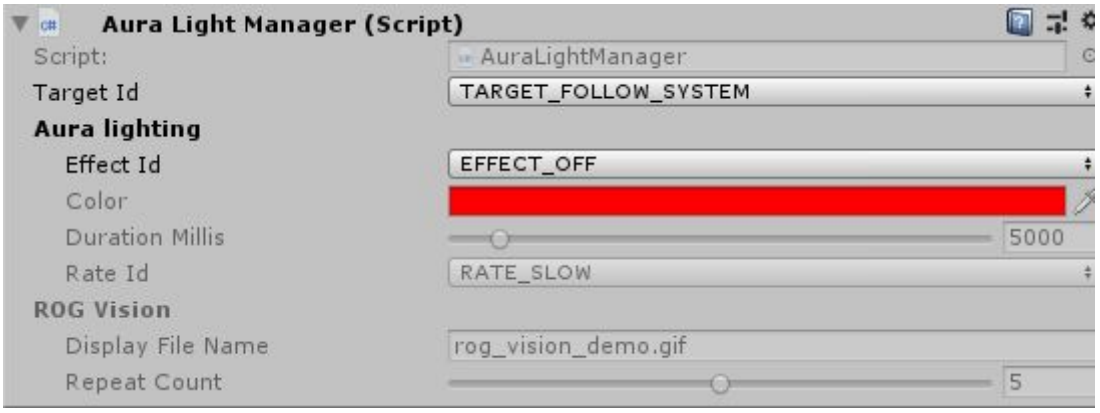
Scripting API: AuraLightManager

Description

Helper class to trigger Aura lighting effect.

Setup

To control Aura lighting, please drag the **AsusAuraLightManager** into your scene.
Click **AsusAuraLightManager** GameObject, and you can see some configurables from your Inspector as follows:



Properties

Color **color**	Color to be used on Aura lighting. Please note that “Solid Black (#000000)” is not supported.
string **displayFileName**	The name of the file that ROG Vision should be displayed.
int **durationMillis**	The number of milliseconds to perform this effect. The maximum duration of an effect is 60 seconds. So the valid range is from 1 to 60000
AuraEffectEnum **effectId**	Aura lighting effect type, it should be one of the following.
AuraRateEnum **rateId**	The flash rate of this effect.
int **repeatCount**	The number of times the file will repeat to show on ROG Vision.

AuraTargetEnum **targetId**	The target where the lighting should be displayed.

Enumerations: AuraEffectEnum

EFFECT_BREATHING	Breathing effect.
EFFECT_COLOR_CYCLE	Cycle effect. Color of this effect can’t be configured.
EFFECT_OFF	Turn off the light.
EFFECT_STATIC	Static effect. Aura lighting will show a static color you choose with specified duration. You can't change the rate of this effect.
EFFECT_STROBING	Strobing effect.

Enumerations: AuraRateEnum

RATE_FAST	Fast flashing rate.
RATE_FAST_1 **RATE_FAST_2** **RATE_FAST_3** **RATE_FAST_4** **RATE_FAST_5** **RATE_FAST_6** **RATE_FAST_7**	RATE_FAST_7 is the fastest rate of the effect.
RATE_MEDIUM	Medium flashing rate.
RATE_SLOW	Slowest flashing rate.

Enumerations: AuraTargetEnum

TARGET_FOLLOW_SYSTEM	Follow system settings to choose lighting target.

TARGET_LOGO	Specify LOGO as lighting target.
TARGET_LOGO_RGB	Specify both LOGO and RGB as lighting target. (for ROG 3)
TARGET_RGB	Specify RGB as lighting target. (for ROG 3)
TARGET_ROG_VISION	Specify ROG Vision as lighting target. (for ROG5)

Public Methods

public int[] **GetAvailableTargetList**()	Get current device supported target list.
public void **StartAuraLight**()	Trigger Aura lighting by current properties.
public void **StartAuraLight**(AuraTargetEnum **targetId** AuraEffectEnum **effectId**, int **color**, AuraRateEnum **rateId**, int **durationMillis**)	Trigger Aura lighting by specific parameter.
public void **StartRogVision**()	Trigger ROG Vision lighting by current properties.
public void **StartRogVision(** string **displayFileName,** int **repeatCount)**	Trigger ROG Vision lighting by specific parameter. Note that the specific file should be placed in the **Assets/StreamingAssets/**

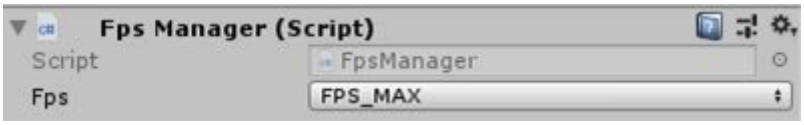
Scripting API: FpsManager

Description

Helper class to modify the display refresh rate.

Setup

To control refresh rate, please drag the **AsusFpsManager** into your scene. We also provide Inspector settings for you.



Once you have added AsusFpsManager into your scene, you can use the following script to control fps.

Properties

FpsEnum **fps**	Frame per second for current device

Enumerations: FpsEnum

FPS_60	60 frames per second.
FPS_90	90 frames per second.
FPS_120	120 frames per second.
FPS_144	144 frames per second.
FPS_MAX	Device supported maximum number of frames per second.

Public Methods

public int[] **GetAvailableFpsList**()	Get current device supported fps list.
public void **SetFps**()	Modify the fps of the current device to properties value.
public void **SetFps**(FpsEnum **fps**)	Modify the fps of the current device to a specific value.

[note]: Please be noted that Fps can't be modified AFTER phone has been plugged into TwinView Dock.

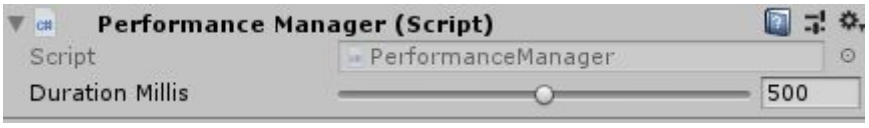
Scripting API: PerformanceManager

Description

Helper class to enable device boosting.

Setup

Our SDK allows you to boost the system performance for a defined period of time. The maximum period of time is one second (i.e. 1,000 ms). To configure, please drag **AsusPerformanceManager** to your scene. And call the following function:



Properties

int **durationMillis**	Time period for device boosting. Valid range is from **1** to **1000**. Ex: When durationMillis is 1000, we'll boost for 1 second.

Public Methods

public void **Boost**()	Boost device by current properties.
public void **Boost**(int **durationMillis**)	Boost device by specific parameters.